

Digital Electronics

Peripherals

Maurizio MARTINA
Electronics and Telecommunications Department
Politecnico di Torino
maurizio.martina@polito.it

1

Disclaimer

Part of the material in this presentation is taken from the
material prepared by:

Prof. F. Gregoretti and Prof. M. Zamboni DET, Polito.

2

Microcontrollers

- Microcontroller (MCU)
 - An integrated electronic computing device which includes three major components on a single chip:
 - Microprocessor (MPU)
 - Memory
 - I/O (Input/Output) ports
 - It usually contains some peripherals:
 - Timers
 - Converters
 - Standard interfaces (I2C, SPI, CAN, USB, Eth, ...)
 - Blocks are connected through a bus

3

Peripherals interface

- The MCU-side interface is made of registers:
 - Control/configuration registers
 - Peripheral enable, mode, speed, ...
 - Status registers
 - Peripheral busy, ready, ...
 - Data registers
 - Input (from MCU to peripheral)
 - Output (from peripheral to MCU)

4

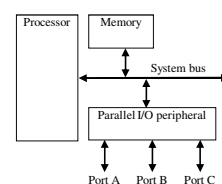
Microprocessor interfacing: I/O addressing

- A microprocessor communicates with other devices using some of its pins:
 - Port-based I/O (parallel I/O)
 - Processor has one or more N-bit ports
 - Processor's software reads and writes a port just like a register
 - E.g.,
 - $P0 = 0xFF$;
 - $v = P1.2$; -- P0 and P1 are 8-bit ports
 - Bus-based I/O
 - Processor has address, data and control ports that form a single bus
 - Communication protocol is built into the processor
 - A single instruction carries out the read or write protocol on the bus

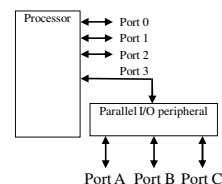
5

I/O extensions

- Parallel I/O peripheral
 - When processor only supports bus-based I/O but parallel I/O needed
 - Each port on peripheral connected to a register within peripheral that is read/written by the processor
- Extended parallel I/O
 - When processor supports port-based I/O but more ports needed
 - One or more processor ports interface with parallel I/O peripheral extending total number of ports available for I/O
 - e.g., extending 4 ports to 6 ports in figure



Adding parallel I/O to a bus-based I/O processor



Extended parallel I/O

6

Memory-mapped and standard I/O

- Processor talks to both memory and peripherals using same bus – two ways to talk to peripherals
 - Memory-mapped I/O
 - Peripheral registers occupy addresses in same address space as memory
 - e.g., Bus has 16-bit address:
 - lower 32k addresses may correspond to memory;
 - upper 32k addresses may correspond to peripherals.
 - Standard I/O (I/O-mapped I/O)
 - Additional pin (*M/IO*) on bus indicates whether a memory or peripheral access.
 - e.g., Bus has 16-bit address
 - all 64K addresses correspond to memory when *M/IO* set to 0;
 - all 64K addresses correspond to peripherals when *M/IO* set to 1.

7

Memory-mapped I/O vs. Standard I/O

- Memory-mapped I/O
 - Requires no special instructions
 - Assembly instructions involving memory like LOAD and STORE work with peripherals as well
 - Standard I/O requires special instructions (e.g., IN, OUT) to move data between peripheral registers and memory
- Standard I/O
 - No loss of memory addresses to peripherals
 - Simpler address decoding logic in peripherals possible
 - When number of peripherals much smaller than address space then high-order address bits can be ignored
 - smaller and/or faster comparators

8

Microprocessor interfacing: interrupts

- Suppose a peripheral intermittently receives data, which must be serviced by the processor
 - The processor can *poll* the peripheral regularly to see if data has arrived;
 - The peripheral can *interrupt* the processor when it has data.
- Requires at least an extra pin (Int):
 - If Int is 1, processor suspends current program and serves the peripheral;
 - Known as interrupt-driven I/O.

9

Interrupt vs. polling

- Polling, also known as time-triggered activation:
 - Is safer and easier to debug, but more wasteful of resources;
 - May be more appropriate for:
 - data-dominated embedded systems (regular streams of sampled data, lots of arithmetic operations, emphasis on throughput);
 - safety-critical embedded systems.
- Interrupt, also known as event-triggered activation:
 - Is harder to debug, but may be more efficient;
 - May be more appropriate for:
 - control-dominated embedded systems (rapid response to irregular events, emphasis on latency)

10

Implementation

- Polling:

```
int main ()
{
    volatile int ready = 0;
    ...
    while (ready != 1)
    {
        ...
    }
    ...
}
```

- Interrupt:

- When the processor receives the interrupt it starts executing a proper routine

- Interrupt Service Routine (ISR)

```
void my_isr (void)
{
    ...
}
```

11

Microprocessor interfacing: interrupts

- What is the address (interrupt address vector) of the ISR?
 - Fixed interrupt (simpler, less flexible)
 - Address built into microprocessor, cannot be changed;
 - If not enough bytes are available the ISR contains a jump.
 - Vectored interrupt (maybe too flexible)
 - Peripheral must provide the address;
 - Common when microprocessor has multiple peripherals connected by a system bus.
 - Compromise: interrupt vector table

12

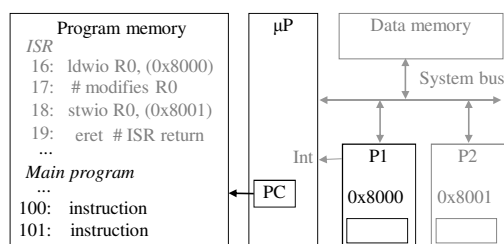
Interrupt vector table

- Compromise between fixed and vectored interrupts
 - One interrupt pin;
 - Table in memory holding ISR addresses;
 - Peripheral doesn't provide ISR address, but only an index into the table:
 - Fewer bits are sent by the peripheral;
 - Can move ISR location without changing peripheral.

13

Interrupt: fixed ISR location - I

1(a): μP is executing its main program

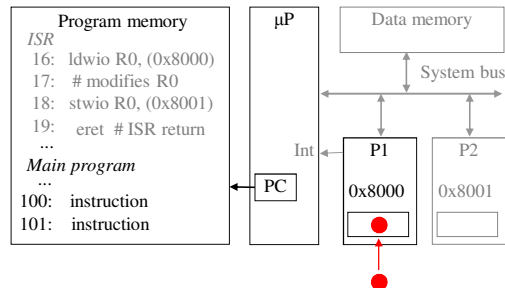


14

Interrupt: fixed ISR location - II

1(a): μP is executing its main program

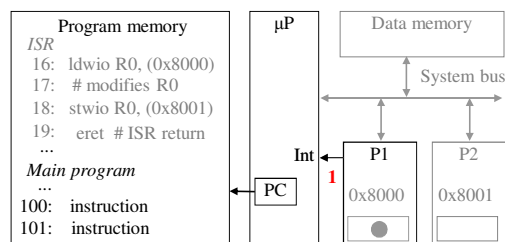
1(b): P1 receives input data in a register with address 0x8000.



15

Interrupt: fixed ISR location - III

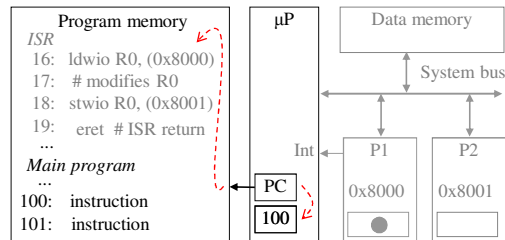
2: P1 asserts *Int* to request servicing by the microprocessor



16

Interrupt: fixed ISR location - IV

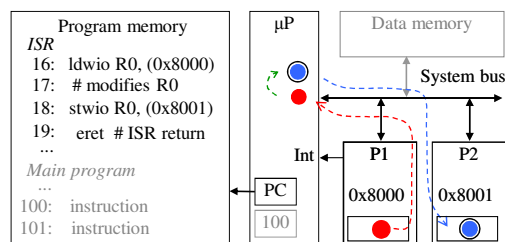
3: After completing instruction at 100, μP sees *Int* asserted, saves the PC's value of 100, and sets PC to the ISR fixed location of 16.



17

Interrupt: fixed ISR location - V

4(a): The ISR reads data from 0x8000, modifies the data, and writes the resulting data to 0x8001.

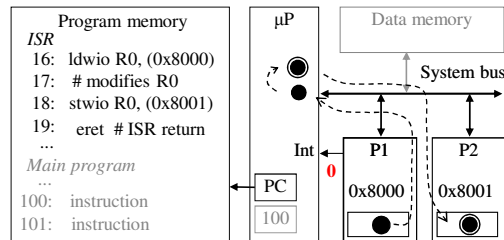


18

Interrupt: fixed ISR location - VI

4(a): The ISR reads data from 0x8000, modifies the data, and writes the resulting data to 0x8001.

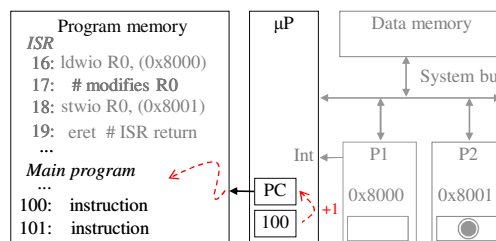
4(b): After being read, P1 deasserts *Int*.



19

Interrupt: fixed ISR location - VII

5: The ISR returns, thus restoring PC to $100+1=101$, where μP resumes executing.

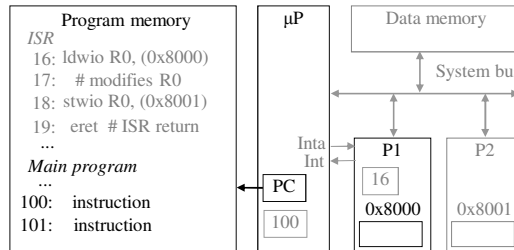


20

Vectored interrupt - I

1(a): P is executing its main program

1(b): P1 receives input data in a register with address 0x8000.

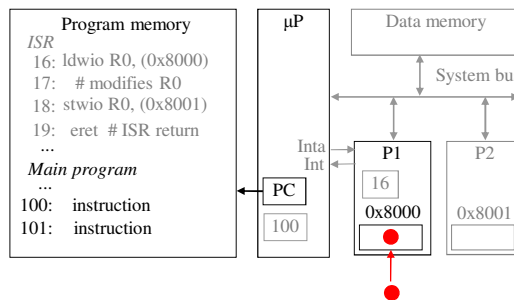


21

Vectored interrupt - II

1(a): P is executing its main program

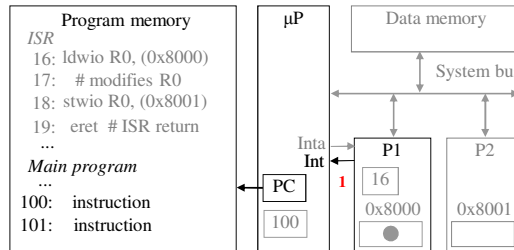
1(b): P1 receives input data in a register with address 0x8000.



22

Vectored interrupt - III

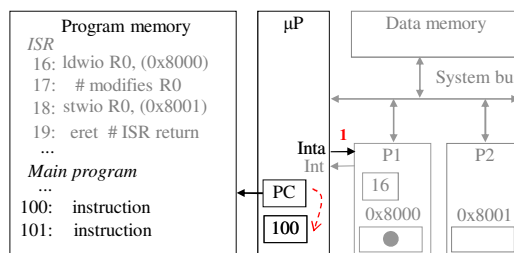
2: P1 asserts *Int* to request servicing by the microprocessor



23

Vectored interrupt - IV

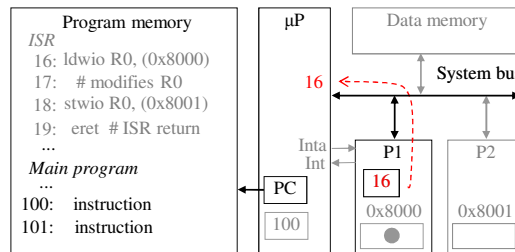
3: After completing instruction at 100, μP sees *Int* asserted, saves the PC's value of 100, and **asserts *Inta***



24

Vectored interrupt - V

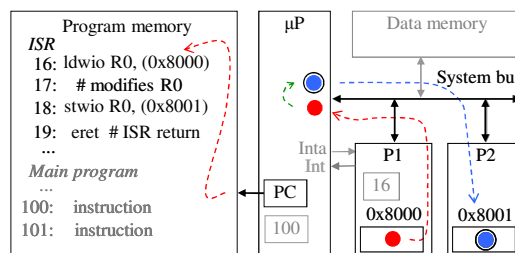
4: P1 detects *Inta* and puts **interrupt address vector 16** on the data bus



25

Vectored interrupt - VI

5(a): PC jumps to the address on the bus (16). The ISR there reads data from 0x8000, modifies the data, and writes the resulting data to 0x8001.

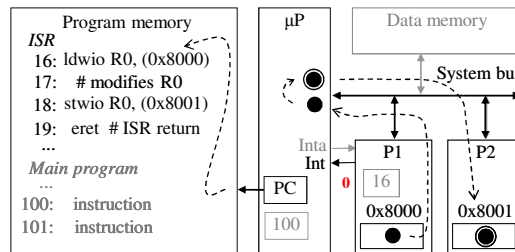


26

Vectored interrupt - VII

5(a): PC jumps to the address on the bus (16). The ISR there reads data from 0x8000, modifies the data, and writes the resulting data to 0x8001.

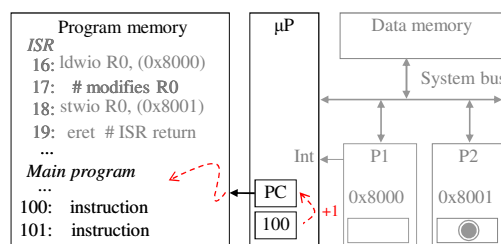
5(b): After being read, P1 deasserts *Int*.



27

Vectored interrupt - VIII

6: The ISR returns, thus restoring the PC to $100+1=101$, where the μP resumes



28

Who saves the processor registers?

- The registers used by the interrupted code must keep the same value when the ISR returns.
- Some processors have a dedicated set of registers, that are used only by ISRs (fast, but with HW cost).
- Other processors automatically save some registers to the stack (flexible, but slow).
- In other cases the ISR itself saves the registers it uses to the stack (flexible and efficient; “RISC-like”).
- Sometimes the compiler reserves some registers for ISRs (useful for simple ISRs which need fast response).

29

Maskable/non-maskable interrupts:

- Some interrupts can be ignored by the MPU (maskable)
- Other interrupts cannot be ignored by the MPU (non-maskable)
 - Reset;
 - Special interrupt pins;
 - Unimplemented opcode;
 - ...

30

Maskable/non-maskable interrupts

- Maskable: programmer can set bit in a register to force processor to ignore interrupt:
 - Important when in the middle of time-critical code;
 - An ISR cannot be interrupted again by the same peripheral.
- Non-maskable: a separate interrupt pin that can't be masked
 - Typically reserved for drastic situations, like reset, power failure requiring immediate backup of data to non-volatile memory.
- Intermediate solution: interrupt priorities
 - An ISR can be interrupted only by a more urgent request, from a peripheral that cannot wait (e.g. the radio packet ISR can interrupt the keyboard ISR, but not vice-versa).

31

Peripheral data managing

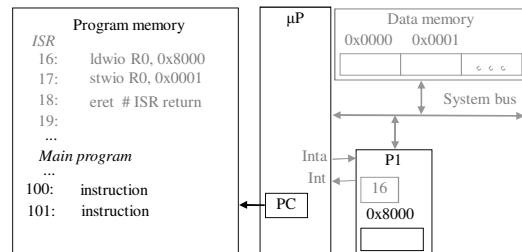
- Some peripheral exchange large amount of data with the MPU.
 - Interrupt:
 - Feasible, but time consuming
 - The MPU “works” for transferring the data
 - Other solutions ?

 - Have a peripheral in charge of moving data to the memory !
 - Data Memory Access (DMA)
 - The MPU can go on with its duties.

32

Data transfer *without* DMA – vect. int. - I

1(a): μP is executing its main program

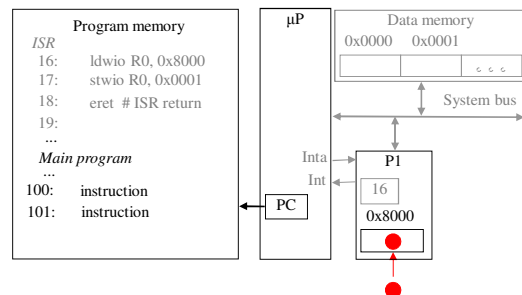


33

Data transfer *without* DMA – vect. int. - II

1(a): μP is executing its main program

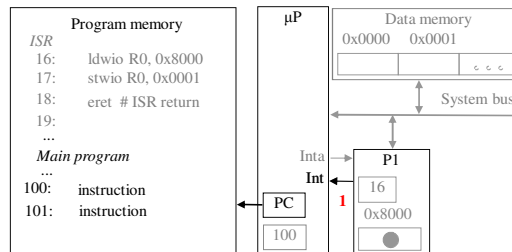
1(b): P1 receives input data in a register with address 0x8000.



34

Data transfer *without* DMA – vect. int. - III

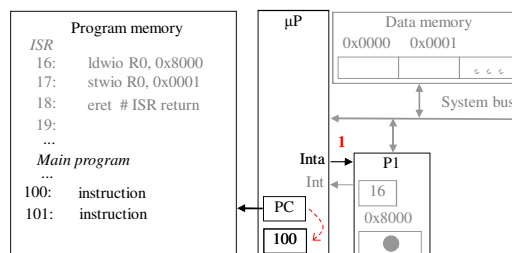
2: P1 asserts *Int* to request servicing by the microprocessor



35

Data transfer *without* DMA – vect. int. - IV

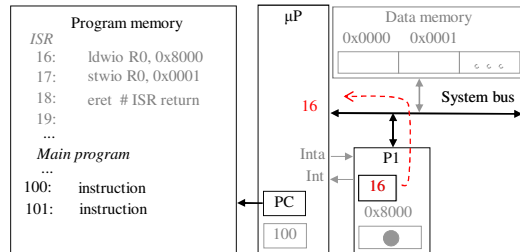
3: After completing instruction at 100, μP sees *Int* asserted, saves the PC's value of 100, and asserts *Inta*.



36

Data transfer *without* DMA – vect. int. - V

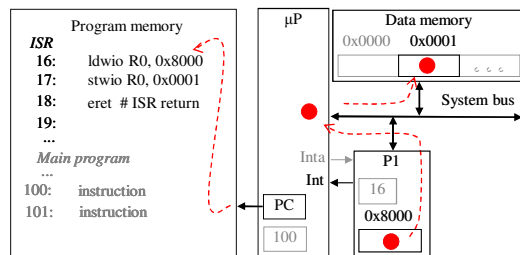
4: P1 detects *Inta* and puts interrupt address vector 16 on the data bus.



37

Data transfer *without* DMA – vect. int. - VI

5(a): μP jumps to the address on the bus (16).
The ISR there reads data from 0x8000 and then writes it to 0x0001, which is in memory.

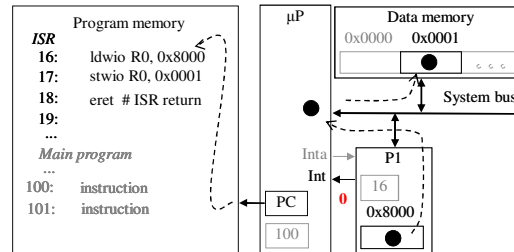


38

Data transfer *without* DMA – vect. int. - VII

5(a): μP jumps to the address on the bus (16).
The ISR there reads data from 0x8000 and then writes it to 0x0001, which is in memory.

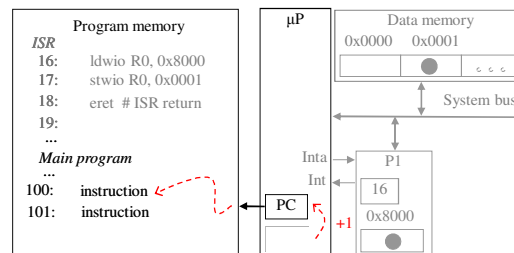
5(b): After being read, P1 de-asserts *Int*.



39

Data transfer *without* DMA – vect. int. - VIII

6: The ISR returns, thus restoring PC to $100+1=101$, where μP resumes executing.



40

Data transfer *with* DMA

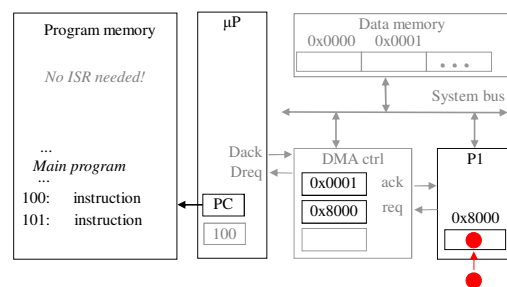
- Before starting the execution of the main task the MPU configures the DMA:
 - Where data have to be take from;
 - Where data have to be copied to.
- Then the MPU can start its main task

41

Data transfer *with* DMA - I

1(a): μP is executing its main program. It has already configured the DMA ctrl registers

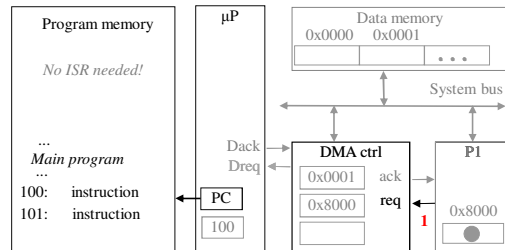
1(b): P1 receives input data in a register with address 0x8000.



42

Data transfer *with* DMA - II

2: P1 asserts *req* to request servicing by DMA ctrl.

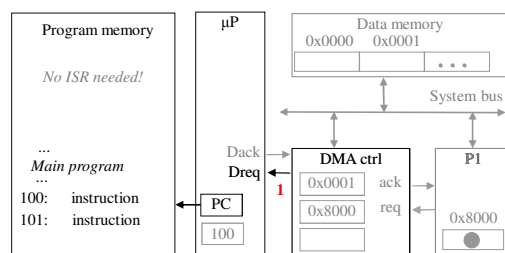


43

Data transfer *with* DMA - III

2: P1 asserts *req* to request servicing by DMA ctrl.

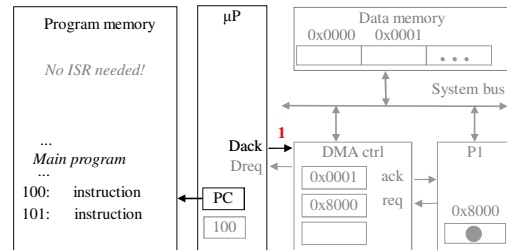
3: DMA ctrl asserts *Dreq* to request control of system bus



44

Data transfer *with* DMA - IV

4: After executing instruction 100, μP sees *Dreq* asserted, releases the system bus, asserts *Dack*, and resumes execution, μP stalls only if it needs the system bus to continue executing.

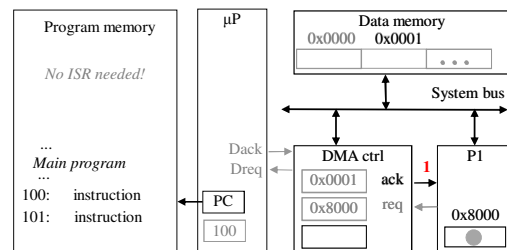


45

Data transfer *with* DMA - V

5: DMA ctrl (a) asserts *ack*, (b) reads data from 0x8000, and (c) writes that data to 0x0001.

(Meanwhile, processor still executing if not stalled!)

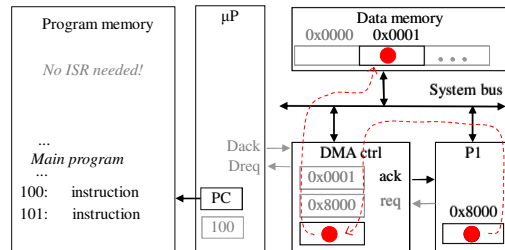


46

Data transfer *with* DMA - VI

5: DMA ctrl (a) asserts ack, (b) reads data from 0x8000, and (c) writes that data to 0x0001.

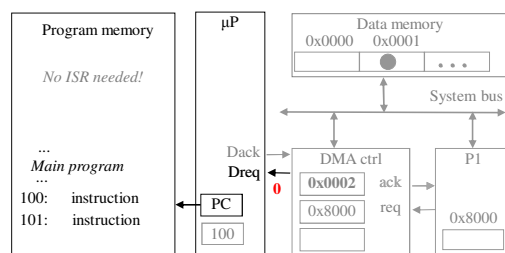
(Meanwhile, processor still executing if not stalled!)



47

Data transfer *with* DMA - VII

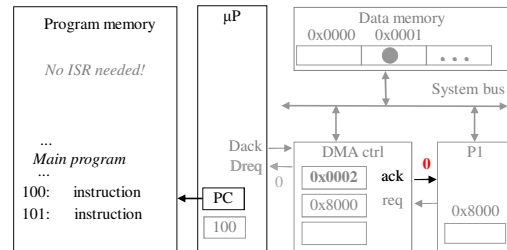
6: DMA de-asserts Dreq.



48

Data transfer *with* DMA - VII

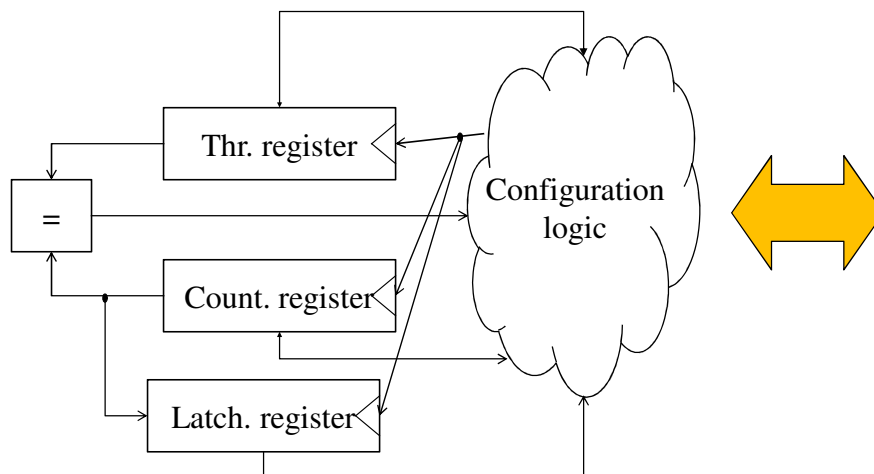
6: Then it de-asserts *ack* completing the handshake with P1, while incrementing the pointer into memory to 0x0002.



49

Timer

- The timer is basically a programmable counter.



50